

Informe técnico - Flujo MCP remoto

Redes

José Luis Gramajo Moraga, Carné 22907

18 de septiembre de 2025 Guatemala, Guatemala

1) Objetivo

Analizar, con evidencia de Wireshark, qué ocurre en cada capa del modelo TCP/IP cuando el chatbot se conecta a un **servidor MCP remoto** vía HTTP (SSE) y ejecuta:

- Handshake MCP (initialize)
- tools/list
- tools/call: echo, dns_lookup, time

2) Entorno de prueba

- **Cliente:** CLI del proyecto (python -m app.main), comandos /remote-*
- **Servidor MCP:** remote_utils (FastMCP con transporte *streamable-http*) montado en /mcp, escuchando en **127.0.0.1:8080**
- **Herramienta:** Wireshark (interfaz **NPF_Loopback**)
- **Captura:** captures/remote_mcp-20250917-0933.pcapng (con copia ...-marked.pcapng / ...-marked.txt)
- **Red:** loopback local (no sale a la red física)

3) Metodología (resumen reproducible)

1. Iniciar el servidor:
python -m remote_utils.server
2. Conectar el cliente:
 - /remote-set http://127.0.0.1:8080/mcp
 - /remote-list
 - /remote-echo hola
 - /remote-dns uvg.edu.gt
 - /remote-time
3. En Wireshark:
 - Filtro base: tcp.port == 8080 && http
 - **Follow TCP stream** sobre cada solicitud y marcar **request/response** relevantes
 - Exportar “marcados” si se requiere anexo

4) Filtros útiles (copiar/pegar en Wireshark)

Handshake & SSE

- Respuestas SSE:
tcp.port == 8080 && http.response.code == 200 && http.content_type contains "event-stream"
- initialize (request):
tcp.port == 8080 && http.request.method == "POST" && frame contains
"\"method\": \"initialize\""
- Limitar al **stream** que estás siguiendo (ejemplos):
tcp.stream == [11|16|22] && tcp.srcport == 8080 && http.content_type contains "event-stream"

tools/list (request)

tcp.port == 8080 && http.request.method == "POST"

&& frame contains "\"method\": \"tools/list\""

tools/call (requests)

- Echo:
tcp.port == 8080 && http.request.method == "POST" && frame contains
"\"method\": \"tools/call\" && frame contains "\"name\": \"echo\""
- DNS:
tcp.port == 8080 && http.request.method == "POST" && frame contains
"\"method\": \"tools/call\" && frame contains "\"name\": \"dns_lookup\""
- Time:
tcp.port == 8080 && http.request.method == "POST" && frame contains
"\"method\": \"tools/call\" && frame contains "\"name\": \"time\""

Respuestas SSE con id (útil para correlacionar)

tcp.port == 8080 && http.content_type contains "event-stream"

&& (http.file_data contains "\"id\":1" or http.file_data contains "\"id\": 1")

5) Evidencia

Fase	Filtro aplicado	Frame (req)	Frame (resp sse)	Tcp.stream	Notas
Initialize	POST /mcp + "method": "initialize"	[271]	[239]	[14]	200 OK + text/event-stream
Tools/list	"method": "tools/list"	[354]	[327]	[16]	Devuelve echo, dns_lookup, time
Call/echo	"method": "tools/call" + "name": "echo"	[198]	(SSE con "id":1) [204]	[11]	result.content[0].text == "hola"
Call/dns	"method": "tools/call" + "name": "dns_lookup"	[317]	(SSE con addresses) [327]	[16]	IPs de uvg.edu.gt
Call/time	"method": "tools/call" + "name": "time"	[438]	(SSE con ISO- UTC) [444]	[22]	Fecha/hora UTC

6) Análisis por capa

6.1 Capa de Enlace (L2)

- **Interfaz Loopback** (Windows). No hay tramas Ethernet reales, **no hay MAC ni ARP**.
- El stack entrega los datagramas directamente a IP/TCP sin pasar por un medio físico.

6.2 Capa de Red (L3)

- **IPv4** 127.0.0.1 → 127.0.0.1 en todos los paquetes.
- **Sin ruteo ni NAT**, no intervienen gateways.
- No se observan fragmentaciones ni ICMP asociados al flujo.

6.3 Capa de Transporte (L4)

- **TCP** hacia **puerto 8080** (servidor), cliente con puertos efímeros (52331, 52325, etc.).
- Conexiones **persistentes** (Keep-Alive). Las respuestas SSE se ven como segmentos **PSH/ACK** pequeños y frecuentes.
- Las respuestas llevan **HTTP/1.1 200 OK** con content-type: text/event-stream y **Transfer-Encoding: chunked** (SSE envía eventos como “trozos”, típicamente líneas que terminan en \n).

6.4 Capa de Aplicación (L7)

- **Protocolo: MCP** (Model Context Protocol) sobre **HTTP/1.1 + Server-Sent Events** (SSE).
- **Mensajería: JSON-RPC 2.0** ("jsonrpc":"2.0","id":N,"method":...), resultados en eventos SSE con data: {...}.

Handshake (initialize)

- Request: {"method":"initialize", "params":{"..."}, "id":0}
- Response (SSE): result con protocolVersion, capabilities, serverInfo (nombre del server y versión).

Listado de herramientas (tools/list)

- Request: {"method":"tools/list", "id":[...]}
- Response: tools:[{name:"echo"}, {name:"dns_lookup"}, {name:"time"}]

Llamadas a herramientas (tools/call)

- **Echo**
 - Req: {"method":"tools/call","params":{"name":"echo","arguments":{"text":"hola"}},"id":1}
 - Resp (SSE): result.content[0].text == "hola"

- **DNS Lookup**
 - Req:


```
{"method":"tools/call","params":{"name":"dns_lookup","arguments":{"host":"uvg.edu.gt"}}, "id":1}
```
 - Resp (SSE): JSON con host y **addresses** (ej.: 45.223.155.41, 45.223.56.41)
- **Time**
 - Req:

```
{"method":"tools/call","params":{"name":"time","arguments":{}},"id":1}
```
 - Resp (SSE): cadena **ISO 8601** en UTC (ej.: 2025-09-17T...Z)

7) Hallazgos clave

- La comunicación MCP remota queda perfectamente reflejada como **HTTP/1.1 + SSE** en Wireshark.
- **Loopback** simplifica el análisis (sin ARP, sin ruteo).
- **SSE** explica por qué las respuestas aparecen como **200 OK (text/event-stream)** y no como una respuesta JSON “tradicional” por cada POST.
- La correlación **request (POST) ↔ response (SSE)** se hace por **tcp.stream** (y, si hace falta, por **"id"** dentro del cuerpo).

8) Limitaciones y notas

- En loopback puede que el **3-way handshake TCP** no se vea en la misma vista filtrada; si lo necesitas, busca `tcp.flags.syn == 1` sin el filtro HTTP.
- En **SSE** el payload va “chunked” y enmarcado por líneas (`event: message, data: ...`), por eso Wireshark muestra paquetes de pocos bytes y saltos de línea (0a).
- Si repites el flujo, los **números de frame y tcp.stream** variarán; actualiza la tabla de evidencia.

9) Conclusiones

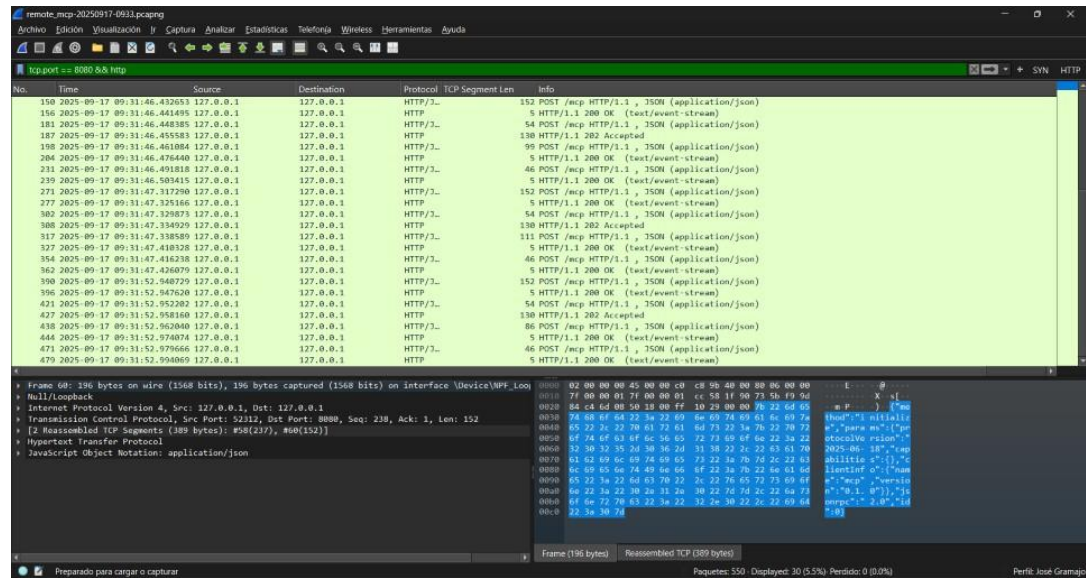
- **MCP potencia y ordena el flujo con herramientas:** comprobar el “handshake” (`initialize`), el inventario (`tools/list`) y las invocaciones (`tools/call`) sobre **HTTP/1.1 + SSE** mostró cómo MCP aporta **determinismo, trazabilidad y separación de responsabilidades** entre chatbot y servidores de funciones.
- **PortHunter MCP aporta valor no trivial:** el análisis de PCAP para identificar patrones de *port scanning*, sospechosos y “primer evento” va **más allá de lo que un LLM puede inferir solo leyendo texto**, al integrar parsing de red y enriquecimiento con fuentes externas (OTX/GreyNoise/GeoLite).
- **La instrumentación funcionó:** las capturas en **loopback** evidencian correctamente la **capa de aplicación** (JSON-RPC/SSE), el **control de transporte** (PSH/ACK y *chunked*) y la correlación por `id/tcp.stream`. Se documentó el flujo completo (`initialize → list → echo/dns/time`).

- **Diseño defensivo:** cuando faltan API keys o base GeoLite, PortHunter devuelve salidas “skipped” con motivo, evitando fallos duros y dejando clara la trazabilidad.

10) Comentarios / Mejoras y trabajo futuro

1. **Seguridad y robustez operativa:** Se implementarán mecanismos de seguridad como TLS, autenticación remota (token o mTLS) y validación de entradas para proteger el plano de control. Se evitará el envío de IPs privadas o sensibles a terceros mediante políticas parametrizables. También se aplicarán estrategias de cacheo, rate limiting y backoff para asegurar un uso responsable de fuentes externas.
2. **Mejora del análisis de red (PortHunter):** Se planea usar parsers reales de PCAP (como pyshark o dpkt) para obtener métricas avanzadas (RTT, entropía, flow graphs, detección de stealth scans, etc.) con soporte completo a IPv6. Además, se incorporarán umbrales adaptativos para clasificar tipos de escaneo, alertas opcionales por email o Slack, y conectores para fuentes externas como AbuseIPDB sin incurrir en costos.
3. **Usabilidad y automatización:** El objetivo es facilitar el análisis con comandos automáticos de exportación de evidencia, perfiles de Wireshark preconfigurados y asistentes CLI paso a paso. Esto permitirá repetir análisis de forma rápida, generar reportes más fácilmente y mejorar la curva de aprendizaje para nuevos usuarios.
4. **Escalabilidad, despliegue y visión futura:** Se facilitará la instalación con Docker Compose y Makefiles para tareas comunes. Se harán pruebas en entornos multi-host y se incluirán PCAPs variados para validar los casos de uso. A futuro, se contempla aplicar el proyecto en honeynets académicas, entornos SOC didácticos y como asistente para respuesta a incidentes con playbooks automatizados.

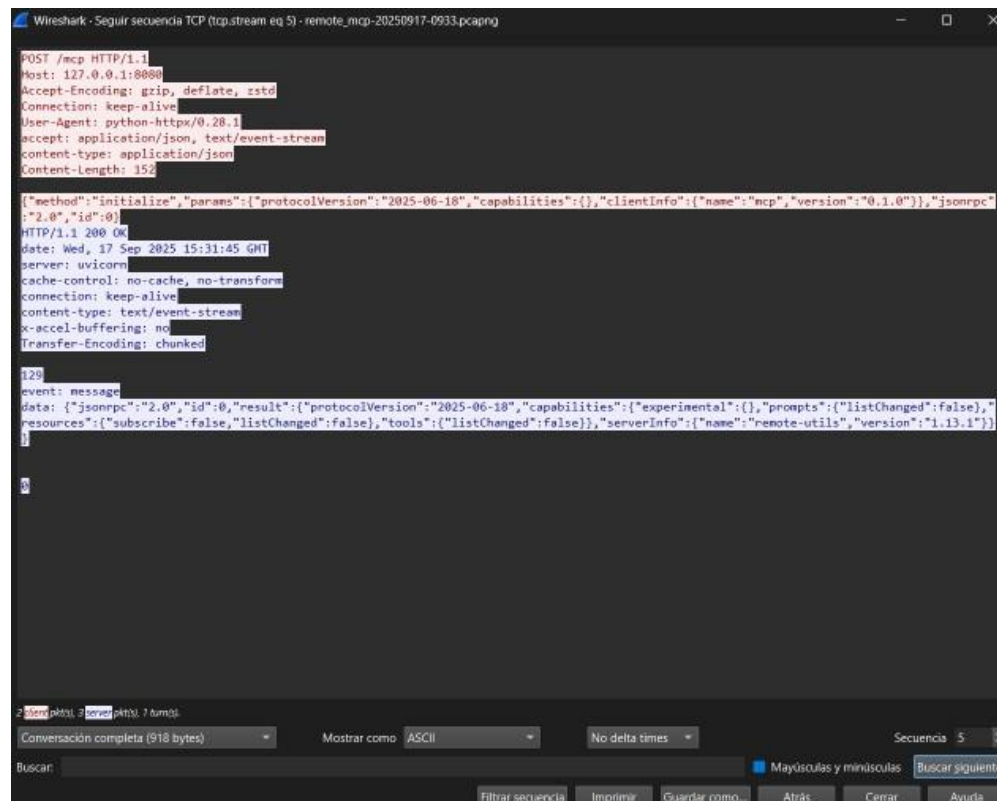
Anexos



The image shows a Wireshark packet capture of an MCP HTTP connection. The top pane displays a list of 479 packets, all of which are HTTP POST requests from 127.0.0.1 to 127.0.0.1. The middle pane shows the details of the selected packet (No. 152), including the Ethernet II, Internet Protocol Version 4, Transmission Control Protocol, and Hypertext Transfer Protocol layers. The bottom pane shows the raw packet data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	TCP Segment Len	Info
150	2025-09-17 09:31:46.432651	127.0.0.1	127.0.0.1	HTTP/3..	152	POST /mcp HTTP/1.1, JSON (application/json)
156	2025-09-17 09:31:46.441495	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
181	2025-09-17 09:31:46.448385	127.0.0.1	127.0.0.1	HTTP/3..	54	POST /mcp HTTP/1.1, JSON (application/json)
187	2025-09-17 09:31:46.455381	127.0.0.1	127.0.0.1	HTTP	130	HTTP/1.1 202 Accepted
198	2025-09-17 09:31:46.461084	127.0.0.1	127.0.0.1	HTTP/3..	99	POST /mcp HTTP/1.1, JSON (application/json)
204	2025-09-17 09:31:46.476440	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
211	2025-09-17 09:31:46.491811	127.0.0.1	127.0.0.1	HTTP/3..	46	POST /mcp HTTP/1.1, JSON (application/json)
219	2025-09-17 09:31:46.503415	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
271	2025-09-17 09:31:47.317290	127.0.0.1	127.0.0.1	HTTP/3..	152	POST /mcp HTTP/1.1, JSON (application/json)
277	2025-09-17 09:31:47.325166	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
302	2025-09-17 09:31:47.328075	127.0.0.1	127.0.0.1	HTTP/3..	54	POST /mcp HTTP/1.1, JSON (application/json)
308	2025-09-17 09:31:47.334920	127.0.0.1	127.0.0.1	HTTP	130	HTTP/1.1 202 Accepted
317	2025-09-17 09:31:47.338589	127.0.0.1	127.0.0.1	HTTP/3..	111	POST /mcp HTTP/1.1, JSON (application/json)
327	2025-09-17 09:31:47.418328	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
354	2025-09-17 09:31:47.418238	127.0.0.1	127.0.0.1	HTTP/3..	46	POST /mcp HTTP/1.1, JSON (application/json)
362	2025-09-17 09:31:47.426070	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
390	2025-09-17 09:31:52.540729	127.0.0.1	127.0.0.1	HTTP/3..	152	POST /mcp HTTP/1.1, JSON (application/json)
396	2025-09-17 09:31:52.547620	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
421	2025-09-17 09:31:52.952202	127.0.0.1	127.0.0.1	HTTP/3..	54	POST /mcp HTTP/1.1, JSON (application/json)
427	2025-09-17 09:31:52.958160	127.0.0.1	127.0.0.1	HTTP	130	HTTP/1.1 202 Accepted
438	2025-09-17 09:31:52.962040	127.0.0.1	127.0.0.1	HTTP/3..	86	POST /mcp HTTP/1.1, JSON (application/json)
444	2025-09-17 09:31:52.974074	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
471	2025-09-17 09:31:52.978666	127.0.0.1	127.0.0.1	HTTP/3..	46	POST /mcp HTTP/1.1, JSON (application/json)
479	2025-09-17 09:31:52.994080	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)

Captura inicial.



The image shows a Wireshark packet capture of an MCP HTTP connection. The top pane displays a list of 479 packets, all of which are HTTP POST requests from 127.0.0.1 to 127.0.0.1. The middle pane shows the details of the selected packet (No. 152), including the Ethernet II, Internet Protocol Version 4, Transmission Control Protocol, and Hypertext Transfer Protocol layers. The bottom pane shows the raw packet data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	TCP Segment Len	Info
150	2025-09-17 09:31:46.432651	127.0.0.1	127.0.0.1	HTTP/3..	152	POST /mcp HTTP/1.1, JSON (application/json)
156	2025-09-17 09:31:46.441495	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
181	2025-09-17 09:31:46.448385	127.0.0.1	127.0.0.1	HTTP/3..	54	POST /mcp HTTP/1.1, JSON (application/json)
187	2025-09-17 09:31:46.455381	127.0.0.1	127.0.0.1	HTTP	130	HTTP/1.1 202 Accepted
198	2025-09-17 09:31:46.461084	127.0.0.1	127.0.0.1	HTTP/3..	99	POST /mcp HTTP/1.1, JSON (application/json)
204	2025-09-17 09:31:46.476440	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
211	2025-09-17 09:31:46.491811	127.0.0.1	127.0.0.1	HTTP/3..	46	POST /mcp HTTP/1.1, JSON (application/json)
219	2025-09-17 09:31:46.503415	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
271	2025-09-17 09:31:47.317290	127.0.0.1	127.0.0.1	HTTP/3..	152	POST /mcp HTTP/1.1, JSON (application/json)
277	2025-09-17 09:31:47.325166	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
302	2025-09-17 09:31:47.328075	127.0.0.1	127.0.0.1	HTTP/3..	54	POST /mcp HTTP/1.1, JSON (application/json)
308	2025-09-17 09:31:47.334920	127.0.0.1	127.0.0.1	HTTP	130	HTTP/1.1 202 Accepted
317	2025-09-17 09:31:47.338589	127.0.0.1	127.0.0.1	HTTP/3..	111	POST /mcp HTTP/1.1, JSON (application/json)
327	2025-09-17 09:31:47.418328	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
354	2025-09-17 09:31:47.418238	127.0.0.1	127.0.0.1	HTTP/3..	46	POST /mcp HTTP/1.1, JSON (application/json)
362	2025-09-17 09:31:47.426070	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
390	2025-09-17 09:31:52.540729	127.0.0.1	127.0.0.1	HTTP/3..	152	POST /mcp HTTP/1.1, JSON (application/json)
396	2025-09-17 09:31:52.547620	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
421	2025-09-17 09:31:52.952202	127.0.0.1	127.0.0.1	HTTP/3..	54	POST /mcp HTTP/1.1, JSON (application/json)
427	2025-09-17 09:31:52.958160	127.0.0.1	127.0.0.1	HTTP	130	HTTP/1.1 202 Accepted
438	2025-09-17 09:31:52.962040	127.0.0.1	127.0.0.1	HTTP/3..	86	POST /mcp HTTP/1.1, JSON (application/json)
444	2025-09-17 09:31:52.974074	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
471	2025-09-17 09:31:52.978666	127.0.0.1	127.0.0.1	HTTP/3..	46	POST /mcp HTTP/1.1, JSON (application/json)
479	2025-09-17 09:31:52.994080	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)

Follow TCP

remote_mcp-20250917-0933.pcapng

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

tcp.port == 8080 && http and !tcp.stream eq 140

Bytes de paquete Cadena "method":"initialize"

Options: Reducido (UTF-8 / ASCII) Mayúsculas y minúsculas Backwards Multiple occurrences

No.	Time	Source	Destination	Protocol	TCP Segment Len	Info
91	2025-09-17 09:31:45.532587	127.0.0.1	127.0.0.1	HTTP/1.1	54	POST /mcp HTTP/1.1, JSON (application/json)
97	2025-09-17 09:31:45.538285	127.0.0.1	127.0.0.1	HTTP	130	HTTP/1.1 202 Accepted
108	2025-09-17 09:31:45.542738	127.0.0.1	127.0.0.1	HTTP/1.1	46	POST /mcp HTTP/1.1, JSON (application/json)
114	2025-09-17 09:31:45.556711	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
150	2025-09-17 09:31:46.432653	127.0.0.1	127.0.0.1	HTTP/1.1	152	POST /mcp HTTP/1.1, JSON (application/json)
156	2025-09-17 09:31:46.441495	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
181	2025-09-17 09:31:46.448385	127.0.0.1	127.0.0.1	HTTP/1.1	54	POST /mcp HTTP/1.1, JSON (application/json)
187	2025-09-17 09:31:46.455583	127.0.0.1	127.0.0.1	HTTP	130	HTTP/1.1 202 Accepted
198	2025-09-17 09:31:46.461884	127.0.0.1	127.0.0.1	HTTP/1.1	99	POST /mcp HTTP/1.1, JSON (application/json)
204	2025-09-17 09:31:46.476448	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
231	2025-09-17 09:31:46.491818	127.0.0.1	127.0.0.1	HTTP/1.1	46	POST /mcp HTTP/1.1, JSON (application/json)
239	2025-09-17 09:31:46.503415	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
302	2025-09-17 09:31:47.329873	127.0.0.1	127.0.0.1	HTTP/1.1	54	POST /mcp HTTP/1.1, JSON (application/json)
308	2025-09-17 09:31:47.334929	127.0.0.1	127.0.0.1	HTTP	130	HTTP/1.1 202 Accepted
317	2025-09-17 09:31:47.338589	127.0.0.1	127.0.0.1	HTTP/1.1	111	POST /mcp HTTP/1.1, JSON (application/json)
327	2025-09-17 09:31:47.410328	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
354	2025-09-17 09:31:47.416238	127.0.0.1	127.0.0.1	HTTP/1.1	46	POST /mcp HTTP/1.1, JSON (application/json)
362	2025-09-17 09:31:47.426079	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
398	2025-09-17 09:31:52.940729	127.0.0.1	127.0.0.1	HTTP/1.1	152	POST /mcp HTTP/1.1, JSON (application/json)
396	2025-09-17 09:31:52.947628	127.0.0.1	127.0.0.1	HTTP	5	HTTP/1.1 200 OK (text/event-stream)
421	2025-09-17 09:31:52.952202	127.0.0.1	127.0.0.1	HTTP/1.1	54	POST /mcp HTTP/1.1, JSON (application/json)
427	2025-09-17 09:31:52.958168	127.0.0.1	127.0.0.1	HTTP	130	HTTP/1.1 202 Accepted

[Next Sequence Number: 1121 (relative sequence number)]
 Acknowledgment Number: 317 (relative ack number)
 Acknowledgment number (raw): 1961872546
 0101 = Header Length: 20 bytes (5)
 Flags: 0x018 (PSH, ACK)
 Window: 254
 [calculated window size: 65924]
 [window size scaling factor: 256]
 Checksum: 0x1867 [unverified]
 [Checksum Status: Unverified]
 Urgent Pointer: 0
 [Timestamps]
 [SEQ/ACK analysis]
 TCP payload (5 bytes)
 TCP segment data (5 bytes)
 A data segment used in reassembly of an upper-layer protocol (RUP) (tcp.segment_data, 5 bytes)

Frame (49 bytes) Reassembled TCP (1120 bytes) De-chunked entity body (888 bytes)

Packets: 350 - Displayed: 26 (5.1%) Marcado: 2 (0.4%) Perdido: 0 (0.0%) Perfil: José Gramajo

Stream TCP con varios POST /mcp y sus HTTP/1.1 200 OK (text/event-stream)